

SoleCount

Build it, run it, put it on GitHub

A beginner's step-by-step guide with all the code

SwiftUI · SwiftData · Swift Charts · StoreKit 2 · iOS 18+
Repo: github.com/KrishnaDistributedcomputing/SoleCount
Prepared for Krishna Venkataraman · 2 November 2026

SoleCount: Build It, Run It, Put It on GitHub

A beginner's step-by-step guide. Follow the steps in order. Don't skip any. Each step tells you what to click or type and what you should see afterwards.

What you're building: SoleCount, an iPhone app that counts how often you wear each pair of sneakers and shows your **cost per wear** (price ÷ number of wears).

Free	SoleCount Pro (one-time $4.99)
Up to 3 pairs	Unlimited pairs
"Wore today" button, wear count, cost per wear	Wear history chart for every pair
Retire a pair, wear limit reminder	Everything in Free

What you need: a Mac with macOS 26.6 or later, Xcode (latest), your iPhone and its cable, and a GitHub account. Your repo already exists at **https://github.com/KrishnaDistributedcomputing/SoleCount**.

How this package is laid out:

```
SoleCount-Package/
  GUIDE.md                <- this guide
  SoleCount-Guide.pdf     <- this guide as a PDF
  CLAUDE.md               <- instructions for Claude Code (goes in your project)
  .gitignore              <- tells Git which files to skip (hidden file!)
  PRIVACY.md              <- your privacy policy (goes in your project)
  README.md               <- front page for your GitHub repo
  SoleCount/              <- all the Swift code files + PrivacyInfo.xcprivacy
```

> **Tip:** In this guide, a grey box like `this` is something you type or paste exactly. **Bold** words are buttons or menus you click.

Part 1: Get the project ready

Step 1. Check your Xcode project (already done)

You already created the project with these settings: **App**, Product Name `SoleCount`, Interface **SwiftUI**, Language **Swift**, Storage **SwiftData**, Testing System **None**, Host in CloudKit **unchecked**, Organization Identifier `com.krishnav`, saved in **Documents**.

Your folders should look like this (open Finder, go to **Documents > SoleCount**):

```
~/Documents/SoleCount/
  SoleCount.xcodeproj      <- double-click this to open the project
```

```
SoleCount/               <- your code lives here
  SoleCountApp.swift
  ContentView.swift
  Item.swift
  Assets.xcassets
```

If the iPhone simulator was still downloading, let it finish first (**Xcode > Settings > Components**). You need it in Part 3.

Step 2. Unzip this package

1. In Finder, double-click **SoleCount-Package.zip** (for example in your **Downloads** folder). A folder called **SoleCount-Package** appears next to it.
2. This guide assumes it is at `~/Downloads/SoleCount-Package`. If you put it somewhere else, change that part of the commands.

Step 3. Delete the template files Xcode made

Xcode's starter app uses an `Item` model we don't need. We'll remove it.

1. Open **SoleCount.xcodeproj** (double-click it in Finder).
2. On the left side of Xcode is the **Project navigator** (the folder icon at the top of the left panel). Click the small arrow next to the yellow/blue **SoleCount** folder to open it.
3. Right-click **Item.swift**, choose **Delete**, then click **Move to Trash**.
4. Right-click **ContentView.swift**, choose **Delete**, then click **Move to Trash**.

Don't delete **SoleCountApp.swift** or **Assets.xcassets**. You'll replace the code inside SoleCountApp.swift in Step 5.

> Xcode will now show red errors because SoleCountApp.swift still mentions `Item` and `ContentView`. That's expected. They go away once all the files are in.

Part 2: Add the code

There are two ways to do this. **Option A is fastest** (copy all the files at once). **Option B** teaches you how to make each file yourself in Xcode. Pick one.

Step 4 (Option A, fastest). Copy all the files with Terminal

1. Quit Xcode (**Xcode > Quit Xcode**, or press **Cmd+Q**).
2. Open **Terminal** (press **Cmd+Space**, type `Terminal`, press **Return**).
3. Paste these lines one at a time, pressing **Return** after each:

```
cd ~/Documents/SoleCount
cp ~/Downloads/SoleCount-Package/SoleCount/*.swift SoleCount/
cp ~/Downloads/SoleCount-Package/SoleCount/PrivacyInfo.xcprivacy SoleCount/
cp ~/Downloads/SoleCount-Package/CLAUDE.md ~/Downloads/SoleCount-Package/PRIVACY.md
~/Downloads/SoleCount-Package/README.md .
cp ~/Downloads/SoleCount-Package/.gitignore .
ls SoleCount
```

4. The last command lists the files. You should see `AddSneakerView.swift`, `Helpers.swift`, `PaywallView.swift`, `PrivacyInfo.xcprivacy`, `SettingsView.swift`, `Sneaker.swift`, `SneakerDetailView.swift`, `SneakerListView.swift`, `SoleCountApp.swift`, `StoreManager.swift`, `WearChartView.swift` and `Assets.xcassets`.
5. Open **SoleCount.xcodeproj** again. New Xcode projects use "synchronized folders", so the files appear in the left panel automatically.

> If a file doesn't appear in Xcode's left panel: right-click the **SoleCount** folder in Xcode, choose **Add Files to "SoleCount"…**, select the missing files, make sure **Target: SoleCount** is checked, and click **Add**.

Then skip to **Step 6**. (Step 5 shows every file's code, so you can read it or use it with Option B.)

Step 4 (Option B). Create each file yourself in Xcode

Do this once for **each file** in Step 5, except SoleCountApp.swift (it already exists, so you just replace its contents):

1. In the left panel, click the **SoleCount** folder (the one with your code).
2. In the menu bar, choose **File > New > File from Template…** (shortcut **Cmd+N**).
3. Make sure **iOS** is selected at the top, click **Swift File**, then **Next**.
4. Type the file name exactly as shown (for example `Sneaker`). Xcode adds `.swift` for you.
5. Check that **Targets: SoleCount** has a tick, and click **Create**.
6. The new file opens with a line like `import Foundation`. Press **Cmd+A** to select everything, press **Delete**, then paste the code from Step 5.

7. Press **Cmd+S** to save.

For **PrivacyInfo.xcprivacy**, use **File > New > File from Template…**, scroll down (or type `privacy` in the filter box), choose **App Privacy**, name it `PrivacyInfo`, and click **Create**. Then right-click the new file, choose **Open As > Source Code**, select everything and paste the code from Step 5.

For **CLAUDE.md**, **PRIVACY.md**, **README.md** and **.gitignore**, copy them with Terminal from the package into `~/Documents/SoleCount` (see the `cp` lines in Option A).

Step 5. The code, file by file

Here is every file with a one-line explanation. Copy each one carefully, from the first line to the last.

5.1 SoleCountApp.swift (replace everything in the existing file)

Starts the app, creates the store for the Pro purchase, and sets up the on-phone database.

```
import SwiftUI
import SwiftData

// The starting point of the app.
// It creates the Store (for the Pro purchase) and the database (SwiftData),
// then shows the list of sneakers.
@main
struct SoleCountApp: App {
    @State private var store = StoreManager()

    var body: some Scene {
        WindowGroup {
            SneakerListView()
                .environment(store)
        }
        .modelContainer(for: Sneaker.self)
    }
}
```

5.2 Sneaker.swift

The "shape" of one pair of sneakers: what gets saved, plus small calculations like cost per wear.

```
import Foundation
import SwiftData

// One pair of sneakers, saved on the phone with SwiftData.
@Model
final class Sneaker {
    var name: String = ""
    var brand: String = ""
    var price: Double = 0
    var purchaseDate: Date = Date.now
    var createdAt: Date = Date.now

    // The photo is stored outside the main database file to keep it fast.
```

```
    @Attribute(.externalStorage) var photoData: Data?

    // Every time you tap "Wore today", the date is added here.
    var wearDates: [Date] = []

    var isRetired: Bool = false

    // Optional: retire the pair after this many wears (nil = no limit).
    var wearLimit: Int?

    init(name: String,
         brand: String,
         price: Double,
         purchaseDate: Date = .now,
         photoData: Data? = nil,
         wearLimit: Int? = nil) {
        self.name = name
        self.brand = brand
        self.price = price
        self.purchaseDate = purchaseDate
        self.createdAt = .now
        self.photoData = photoData
        self.wearDates = []
        self.isRetired = false
        self.wearLimit = wearLimit
    }

    // MARK: - Handy calculations (not saved, worked out on the fly)

    var wearCount: Int { wearDates.count }

    /// Price divided by number of wears. nil until the first wear.
    var costPerWear: Double? {
        wearCount > 0 ? price / Double(wearCount) : nil
    }

    var woreToday: Bool {
        wearDates.contains { Calendar.current.isDateInToday($0) }
    }

    var hasReachedLimit: Bool {
        guard let limit = wearLimit, limit > 0 else { return false }
        return wearCount >= limit
    }

    func logWear(on date: Date = .now) {
        wearDates.append(date)
    }

    func undoTodayWear() {
        if let index = wearDates.lastIndex(where: { Calendar.current.isDateInToday($0) }) {
            wearDates.remove(at: index)
        }
    }
}
```

5.3 Helpers.swift

Money formatting, photo shrinking, and two small reusable views (the photo square and the stat box).

```
import SwiftUI
import UIKit

// MARK: - Money formatting

extension Double {
    /// Shows a number as money in the phone's currency, e.g. "$12.50".
    var asCurrency: String {
        formatted(.currency(code: Locale.current.currency?.identifier ?? "USD"))
    }
}

// MARK: - Photo shrinking

enum ImageHelper {
    /// Shrinks a photo to at most 1024 pixels wide/tall and saves it as JPEG,
    /// so the app stays small and fast.
    static func downsized(_ data: Data, maxDimension: CGFloat = 1024) -> Data? {
        guard let image = UIImage(data: data) else { return nil }
        let size = image.size
        let longestSide = max(size.width, size.height)
        guard longestSide > 0 else { return nil }
        let scale = min(1, maxDimension / longestSide)
        let newSize = CGSize(width: size.width * scale, height: size.height * scale)

        let format = UIGraphicsImageRendererFormat.default()
        format.scale = 1
        let renderer = UIGraphicsImageRenderer(size: newSize, format: format)
        let resized = renderer.image { _ in
            image.draw(in: CGRect(origin: .zero, size: newSize))
        }
        return resized.jpegData(compressionQuality: 0.8)
    }
}

// MARK: - Reusable little views

/// A square photo of the sneaker, or a shoe icon if there is no photo.
struct SneakerPhoto: View {
    let data: Data?
    var size: CGFloat = 56

    var body: some View {
        Group {
            if let data, let uiImage = UIImage(data: data) {
                Image(uiImage: uiImage)
                    .resizable()
                    .scaledToFill()
            } else {
                ZStack {
                    Rectangle().fill(Color.secondary.opacity(0.15))
                    Image(systemName: "shoe.fill")
                        .font(.system(size: size * 0.4))
```

```
                        .foregroundStyle(.secondary)
                }
            }
        }
        .frame(width: size, height: size)
        .clipShape(RoundedRectangle(cornerRadius: size * 0.2))
        .accessibilityHidden(true)
    }
}

/// A small box with a big number and a label underneath.
struct StatCard: View {
    let title: String
    let value: String

    var body: some View {
        VStack(spacing: 4) {
            Text(value)
                .font(.title3.bold())
                .lineLimit(1)
                .minimumScaleFactor(0.6)
            Text(title)
                .font(.caption)
                .foregroundStyle(.secondary)
        }
        .frame(maxWidth: .infinity)
        .padding(.vertical, 12)
        .background(.thinMaterial, in: RoundedRectangle(cornerRadius: 12))
        .accessibilityElement(children: .combine)
    }
}
```

5.4 SneakerListView.swift

The home screen: your pairs, their wears and cost per wear, and the **+** button that checks the free 3-pair limit.

```
import SwiftUI
import SwiftData

// The first screen: all your sneakers, with wears and cost per wear.
struct SneakerListView: View {
    @Environment(\.modelContext) private var modelContext
    @Environment(StoreManager.self) private var store
    @Query(sort: \Sneaker.createdAt, order: .reverse) private var sneakers: [Sneaker]

    @State private var showingAdd = false
    @State private var showingPaywall = false
    @State private var showingSettings = false

    private var activeSneakers: [Sneaker] { sneakers.filter { !$0.isRetired } }
    private var retiredSneakers: [Sneaker] { sneakers.filter { $0.isRetired } }

    var body: some View {
        NavigationStack {
            Group {
                if sneakers.isEmpty {
                    ContentUnavailableView {
```

```
                        Label("No sneakers yet", systemImage: "shoe.2")
                    } description: {
                        Text("Add your first pair and tap \"Wore today\" whenever you wear them.")
                    } actions: {
                        Button("Add a pair") { addTapped() }
                            .buttonStyle(.borderedProminent)
                    }
                } else {
                    List {
                        if !activeSneakers.isEmpty {
                            Section("In rotation") {
                                ForEach(activeSneakers) { sneaker in
                                    NavigationLink(value: sneaker) {
                                        SneakerRow(sneaker: sneaker)
                                    }
                                }
                                .onDelete { offsets in delete(offsets, from: activeSneakers) }
                            }
                        }

                        if !retiredSneakers.isEmpty {
                            Section("Retired") {
                                ForEach(retiredSneakers) { sneaker in
                                    NavigationLink(value: sneaker) {
                                        SneakerRow(sneaker: sneaker)
                                    }
                                }
                                .onDelete { offsets in delete(offsets, from: retiredSneakers) }
                            }
                        }

                        if !store.isPro {
                            Section {
                                Button {
                                    showingPaywall = true
                                } label: {
                                    Label("Unlock SoleCount Pro", systemImage: "star.fill")
                                }
                            } footer: {
                                Text("The free version tracks up to \(StoreManager.freePairLimit) pairs. Swipe left on a pair to delete it.")
                            }
                        }
                    }
                }
            }
            .navigationTitle("SoleCount")
            .navigationDestination(for: Sneaker.self) { sneaker in
                SneakerDetailView(sneaker: sneaker)
            }
            .toolbar {
                ToolbarItem(placement: .topBarLeading) {
                    Button {
                        showingSettings = true
                    } label: {
                        Image(systemName: "gearshape")
                    }
                    .accessibilityLabel("Settings")
```

```
                }
                ToolbarItem(placement: .topBarTrailing) {
                    Button {
                        addTapped()
                    } label: {
                        Image(systemName: "plus")
                    }
                    .accessibilityLabel("Add sneaker")
                }
            }
            .sheet(isPresented: $showingAdd) { AddSneakerView() }
            .sheet(isPresented: $showingPaywall) { PaywallView() }
            .sheet(isPresented: $showingSettings) { SettingsView() }
        }
    }

    /// Free users can add up to 3 pairs. After that we show the Pro screen.
    private func addTapped() {
        if store.isPro || sneakers.count < StoreManager.freePairLimit {
            showingAdd = true
        } else {
            showingPaywall = true
        }
    }

    private func delete(_ offsets: IndexSet, from list: [Sneaker]) {
        for index in offsets {
            modelContext.delete(list[index])
        }
    }
}

/// One row in the list.
struct SneakerRow: View {
    let sneaker: Sneaker

    var body: some View {
        HStack(spacing: 12) {
            SneakerPhoto(data: sneaker.photoData, size: 56)
            VStack(alignment: .leading, spacing: 2) {
                Text(sneaker.name)
                    .font(.headline)
                if !sneaker.brand.isEmpty {
                    Text(sneaker.brand)
                        .font(.subheadline)
                        .foregroundStyle(.secondary)
                }
            }
            Spacer()
            VStack(alignment: .trailing, spacing: 2) {
                Text("\(sneaker.wearCount) wears")
                    .font(.subheadline.bold())
                Text(sneaker.costPerWear.map { "\($0.asCurrency)/wear" } ?? "Not worn yet")
                    .font(.caption)
                    .foregroundStyle(.secondary)
            }
        }
        .padding(.vertical, 4)
```

```
    }
}
```

5.5 AddSneakerView.swift

The "New Sneaker" form with a photo picker, name, brand, price, purchase date and optional wear limit.

```
import SwiftUI
import SwiftData
import PhotosUI

// The "New Sneaker" form.
struct AddSneakerView: View {
    @Environment(\.modelContext) private var modelContext
    @Environment(\.dismiss) private var dismiss

    @State private var name = ""
    @State private var brand = ""
    @State private var price: Double?
    @State private var purchaseDate = Date.now
    @State private var hasWearLimit = false
    @State private var wearLimit = 100
    @State private var photoItem: PhotosPickerItem?
    @State private var photoData: Data?

    private var canSave: Bool {
        !name.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty
    }

    var body: some View {
        NavigationStack {
            Form {
                Section {
                    HStack {
                        Spacer()
                        PhotosPicker(selection: $photoItem, matching: .images) {
                            VStack(spacing: 8) {
                                SneakerPhoto(data: photoData, size: 120)
                                Text(photoData == nil ? "Add photo" : "Change photo")
                                    .font(.footnote)
                            }
                        }
                        Spacer()
                    }
                }
                .listRowBackground(Color.clear)

                Section("Details") {
                    TextField("Name (e.g. Air Max 90)", text: $name)
                    TextField("Brand (e.g. Nike)", text: $brand)
                    TextField("Price paid (e.g. 129.99)", value: $price, format: .number)
                        .keyboardType(.decimalPad)
                    DatePicker("Purchased",
                               selection: $purchaseDate,
                               in: ...Date.now,
                               displayedComponents: .date)
                }
```

```
                Section {
                    Toggle("Set a wear limit", isOn: $hasWearLimit)
                    if hasWearLimit {
                        Stepper("Retire after \(wearLimit) wears",
                                value: $wearLimit, in: 10...1000, step: 10)
                    }
                } footer: {
                    Text("Optional. SoleCount will suggest retiring the pair when it reaches this
many wears.")
                }
            }
            .navigationTitle("New Sneaker")
            .navigationBarTitleDisplayMode(.inline)
            .toolbar {
                ToolbarItem(placement: .cancellationAction) {
                    Button("Cancel") { dismiss() }
                }
                ToolbarItem(placement: .confirmationAction) {
                    Button("Save") { save() }
                        .disabled(!canSave)
                }
            }
            .onChange(of: photoItem) { _, newItem in
                Task { await loadPhoto(from: newItem) }
            }
        }
    }

    private func loadPhoto(from item: PhotosPickerItem?) async {
        guard let item,
              let data = try? await item.loadTransferable(type: Data.self) else { return }
        photoData = ImageHelper.downsized(data)
    }

    private func save() {
        let sneaker = Sneaker(
            name: name.trimmingCharacters(in: .whitespacesAndNewlines),
            brand: brand.trimmingCharacters(in: .whitespacesAndNewlines),
            price: max(0, price ?? 0),
            purchaseDate: purchaseDate,
            photoData: photoData,
            wearLimit: hasWearLimit ? wearLimit : nil
        )
        modelContext.insert(sneaker)
        dismiss()
    }
}
```

5.6 SneakerDetailView.swift

One pair's screen: the big **Wore today** button, stats, undo, retire, and the chart (Pro).

```
import SwiftUI
import SwiftData

// The screen for one pair: big "Wore today" button, stats, chart, retire.
```

```
struct SneakerDetailView: View {
    @Environment(StoreManager.self) private var store
    @Bindable var sneaker: Sneaker

    @State private var showingPaywall = false
    @State private var showingLimitAlert = false

    var body: some View {
        ScrollView {
            VStack(spacing: 20) {
                SneakerPhoto(data: sneaker.photoData, size: 200)

                VStack(spacing: 4) {
                    Text(sneaker.name)
                        .font(.title.bold())
                        .multilineTextAlignment(.center)
                    if !sneaker.brand.isEmpty {
                        Text(sneaker.brand)
                            .foregroundStyle(.secondary)
                    }
                    if sneaker.isRetired {
                        Label("Retired", systemImage: "archivebox.fill")
                            .font(.subheadline.bold())
                            .foregroundStyle(.orange)
                            .padding(.top, 4)
                    }
                }

                HStack(spacing: 12) {
                    StatCard(title: "Wears", value: "\(sneaker.wearCount)")
                    StatCard(title: "Cost per wear", value: sneaker.costPerWear?.asCurrency ??
"—")
                    StatCard(title: "Price", value: sneaker.price.asCurrency)
                }

                if let limit = sneaker.wearLimit, limit > 0 {
                    ProgressView(value: Double(min(sneaker.wearCount, limit)), total:
Double(limit)) {
                        Text("\(sneaker.wearCount) of \(limit) wears")
                            .font(.subheadline)
                    }
                }

                if !sneaker.isRetired {
                    Button {
                        logWear()
                    } label: {
                        Label(sneaker.woreToday ? "Wore them again" : "Wore today",
                              systemImage: "checkmark.circle.fill")
                            .font(.title2.bold())
                            .frame(maxWidth: .infinity)
                            .padding(.vertical, 14)
                    }
                    .buttonStyle(.borderedProminent)
                    .controlSize(.large)
                    .sensoryFeedback(.success, trigger: sneaker.wearCount)

                    if sneaker.woreToday {
```

```
                    Button("Undo today's wear", role: .destructive) {
                        sneaker.undoTodayWear()
                    }
                    .font(.footnote)
                }
            }

            chartSection
        }
        .padding()
    }
    .navigationTitle(sneaker.name)
    .navigationBarTitleDisplayMode(.inline)
    .toolbar {
        ToolbarItem(placement: .topBarTrailing) {
            Button(sneaker.isRetired ? "Unretire" : "Retire") {
                sneaker.isRetired.toggle()
            }
        }
    }
    .alert("Time to retire this pair?", isPresented: $showingLimitAlert) {
        Button("Retire now") { sneaker.isRetired = true }
        Button("Keep wearing", role: .cancel) { }
    } message: {
        Text("You've reached the wear limit you set for this pair.")
    }
    .sheet(isPresented: $showingPaywall) {
        PaywallView()
    }
}

@ViewBuilder
private var chartSection: some View {
    VStack(alignment: .leading, spacing: 8) {
        Text("Wear history (last 6 months)")
            .font(.headline)

        if store.isPro {
            WearChartView(wearDates: sneaker.wearDates)
        } else {
            Button {
                showingPaywall = true
            } label: {
                HStack {
                    Image(systemName: "lock.fill")
                    Text("Unlock the wear history chart with Pro")
                }
                .frame(maxWidth: .infinity)
                .padding()
                .background(.thinMaterial, in: RoundedRectangle(cornerRadius: 12))
            }
            .buttonStyle(.plain)
        }
    }
    .frame(maxWidth: .infinity, alignment: .leading)
}

private func logWear() {
```

```
        sneaker.logWear()
        if let limit = sneaker.wearLimit, limit > 0, sneaker.wearCount == limit {
            showingLimitAlert = true
        }
    }
}
```

5.7 WearChartView.swift

A bar chart of wears per month for the last 6 months, using Apple's Swift Charts.

```
import SwiftUI
import Charts

// Bar chart: how many times you wore the pair in each of the last 6 months.
struct WearChartView: View {
    let wearDates: [Date]

    private struct MonthCount: Identifiable {
        let month: Date
        let count: Int
        var id: Date { month }
    }

    private var data: [MonthCount] {
        let calendar = Calendar.current
        let thisMonth = calendar.dateInterval(of: .month, for: .now)?.start ?? .now
        var result: [MonthCount] = []
        for offset in stride(from: 5, through: 0, by: -1) {
            guard let start = calendar.date(byAdding: .month, value: -offset, to: thisMonth) else
{ continue }
            let count = wearDates.filter {
                calendar.isDate($0, equalTo: start, toGranularity: .month)
            }.count
            result.append(MonthCount(month: start, count: count))
        }
        return result
    }

    var body: some View {
        Chart(data) { item in
            BarMark(
                x: .value("Month", item.month, unit: .month),
                y: .value("Wears", item.count)
            )
            .foregroundStyle(Color.accentColor.gradient)
            .cornerRadius(4)
        }
        .chartXAxis {
            AxisMarks(values: .stride(by: .month)) { _ in
                AxisValueLabel(format: .dateTime.month(.abbreviated))
            }
        }
        .frame(height: 200)
        .accessibilityLabel("Wear history chart")
```

```
    }
}
```

5.8 StoreManager.swift

Handles the one-time Pro purchase and **Restore Purchases** with Apple's StoreKit 2. No server needed.

```
import Foundation
import StoreKit
import Observation

// Handles the one-time "SoleCount Pro" purchase with StoreKit 2.
// No servers needed: Apple's StoreKit checks the purchase on the phone.
@MainActor
@Observable
final class StoreManager {
    static let proProductID = "com.krishnav.SoleCount.pro"
    static let freePairLimit = 3
    private static let isProKey = "isPro"

    private(set) var proProduct: Product?
    private(set) var isPro: Bool = UserDefaults.standard.bool(forKey: StoreManager.isProKey)
    private(set) var isLoading = false

    // Used to show an error alert.
    var showError = false
    private(set) var errorMessage = ""

    @ObservationIgnored private var updatesTask: Task<Void, Never>?

    init() {
        // Listen for purchases that happen outside the app
        // (e.g. Ask to Buy approved later, refunds, purchases on another device).
        updatesTask = Task { [weak self] in
            for await result in Transaction.updates {
                await self?.handle(result)
            }
        }
        Task { await refresh() }
    }

    func refresh() async {
        await loadProducts()
        await updateEntitlements()
    }

    func loadProducts() async {
        do {
            let products = try await Product.products(for: [Self.proProductID])
            proProduct = products.first
        } catch {
            fail("Couldn't load the App Store. Check your internet connection and try again.")
        }
    }

    /// Asks StoreKit what this Apple Account owns.
    func updateEntitlements() async {
```

```
        var ownsPro = false
        for await result in Transaction.currentEntitlements {
            if case .verified(let transaction) = result,
               transaction.productID == Self.proProductID,
               transaction.revocationDate == nil {
                ownsPro = true
            }
        }
        setPro(ownsPro)
    }

    func purchase() async {
        guard let product = proProduct else {
            await loadProducts()
            return
        }
        isLoading = true
        defer { isLoading = false }

        do {
            let result = try await product.purchase()
            switch result {
            case .success(let verification):
                switch verification {
                case .verified(let transaction):
                    setPro(true)
                    await transaction.finish()
                case .unverified:
                    fail("We couldn't verify this purchase. Please try again.")
                }
            case .pending:
                fail("Your purchase is waiting for approval (for example Ask to Buy). Pro will
unlock automatically once it's approved.")
            case .userCancelled:
                break
            @unknown default:
                break
            }
        } catch {
            fail("The purchase didn't go through: \(error.localizedDescription)")
        }
    }

    /// Required by App Review: lets people get Pro back on a new phone.
    func restore() async {
        isLoading = true
        defer { isLoading = false }

        do {
            try await AppStore.sync()
        } catch {
            // The user may have cancelled the sign-in prompt; still check below.
        }
        await updateEntitlements()
        if !isPro {
            fail("No previous SoleCount Pro purchase was found for this Apple Account.")
        }
    }
```

```
    private func handle(_ result: VerificationResult<Transaction>) async {
        if case .verified(let transaction) = result {
            await transaction.finish()
        }
        await updateEntitlements()
    }

    private func setPro(_ value: Bool) {
        isPro = value
        UserDefaults.standard.set(value, forKey: Self.isProKey)
    }

    private func fail(_ message: String) {
        errorMessage = message
        showError = true
    }
}
```

5.9 PaywallView.swift

The SoleCount Pro screen. It shows the real price from the App Store, a Buy button, Restore Purchases, and Privacy and Terms links, which is what App Review expects.

```
import SwiftUI
import StoreKit

// The "SoleCount Pro" screen with Buy and Restore buttons.
struct PaywallView: View {
    @Environment(StoreManager.self) private var store
    @Environment(\.dismiss) private var dismiss

    var body: some View {
        @Bindable var store = store

        NavigationStack {
            ScrollView {
                VStack(spacing: 22) {
                    Image(systemName: "shoe.2.fill")
                        .font(.system(size: 64))
                        .foregroundStyle(.tint)
                        .padding(.top, 24)

                    Text("SoleCount Pro")
                        .font(.largeTitle.bold())

                    Text("One-time purchase. No subscription.")
                        .foregroundStyle(.secondary)

                    VStack(alignment: .leading, spacing: 14) {
                        FeatureRow(icon: "infinity", text: "Track unlimited pairs (free: \
(StoreManager.freePairLimit))")
                        FeatureRow(icon: "chart.bar.fill", text: "Wear history chart for every
pair")
                        FeatureRow(icon: "heart.fill", text: "Support an independent developer")
                    }
```

```
                    .frame(maxWidth: .infinity, alignment: .leading)
                    .padding()
                    .background(.thinMaterial, in: RoundedRectangle(cornerRadius: 16))

                    if store.isPro {
                        Label("You have Pro. Thank you!", systemImage: "checkmark.seal.fill")
                            .font(.headline)
                            .foregroundStyle(.green)
                    } else {
                        Button {
                            Task {
                                await store.purchase()
                                if store.isPro { dismiss() }
                            }
                        } label: {
                            Group {
                                if store.isLoading {
                                    ProgressView()
                                } else {
                                    Text(buyButtonTitle)
                                }
                            }
                            .font(.headline)
                            .frame(maxWidth: .infinity)
                            .padding(.vertical, 6)
                        }
                        .buttonStyle(.borderedProminent)
                        .controlSize(.large)
                        .disabled(store.proProduct == nil || store.isLoading)

                        if store.proProduct == nil {
                            Button("Try loading the price again") {
                                Task { await store.loadProducts() }
                            }
                            .font(.footnote)
                        }
                    }

                    Button("Restore Purchases") {
                        Task {
                            await store.restore()
                            if store.isPro { dismiss() }
                        }
                    }
                    .disabled(store.isLoading)

                    Text("A one-time payment charged to your Apple Account. Pro stays unlocked on devices signed in to the same Apple Account. Use Restore Purchases on a new device.")
                        .font(.footnote)
                        .foregroundStyle(.secondary)
                        .multilineTextAlignment(.center)

                    HStack(spacing: 16) {
                        Link("Privacy Policy", destination: AppLinks.privacyPolicy)
                        Link("Terms of Use", destination: AppLinks.termsOfUse)
                    }
                    .font(.footnote)
                }
```

```
                .padding()
            }
            .toolbar {
                ToolbarItem(placement: .cancellationAction) {
                    Button("Close") { dismiss() }
                }
            }
            .task {
                if store.proProduct == nil { await store.loadProducts() }
            }
            .alert("Store", isPresented: $store.showError) {
                Button("OK", role: .cancel) { }
            } message: {
                Text(store.errorMessage)
            }
        }
    }

    private var buyButtonTitle: String {
        if let product = store.proProduct {
            return "Unlock Pro for \(product.displayPrice)"
        }
        return "Loading price…"
    }
}

struct FeatureRow: View {
    let icon: String
    let text: String

    var body: some View {
        HStack(spacing: 12) {
            Image(systemName: icon)
                .font(.title3)
                .foregroundStyle(.tint)
                .frame(width: 30)
            Text(text)
        }
    }
}
```

5.10 SettingsView.swift

The gear-icon screen: Pro status, Restore Purchases, Privacy Policy, Support and app version.

```
import SwiftUI

// Links you can change in one place.
// The privacy policy page must be public for App Review.
enum AppLinks {
    static let privacyPolicy = URL(string:
"https://github.com/KrishnaDistributedcomputing/SoleCount/blob/main/PRIVACY.md")!
    // Apple's standard license agreement (fine to use for one-time purchases).
    static let termsOfUse = URL(string: "https://www.apple.com/legal/internet-
services/itunes/dev/stdeula/")!
    static let support = URL(string:
"https://github.com/KrishnaDistributedcomputing/SoleCount/issues")!
```

```
}

// The gear-icon screen: Pro status, Restore Purchases, privacy, version.
struct SettingsView: View {
    @Environment(StoreManager.self) private var store
    @Environment(\.dismiss) private var dismiss
    @State private var showingPaywall = false

    private var appVersion: String {
        let version = Bundle.main.infoDictionary?["CFBundleShortVersionString"] as? String ??
"1.0"
        let build = Bundle.main.infoDictionary?["CFBundleVersion"] as? String ?? "1"
        return "\(version) (\(build))"
    }

    var body: some View {
        @Bindable var store = store

        NavigationStack {
            List {
                Section("SoleCount Pro") {
                    if store.isPro {
                        Label("Pro unlocked", systemImage: "checkmark.seal.fill")
                            .foregroundStyle(.green)
                    } else {
                        Button {
                            showingPaywall = true
                        } label: {
                            Label("Unlock Pro", systemImage: "star.fill")
                        }
                    }
                    Button("Restore Purchases") {
                        Task { await store.restore() }
                    }
                    .disabled(store.isLoading)
                }

                Section("About") {
                    Link("Privacy Policy", destination: AppLinks.privacyPolicy)
                    Link("Terms of Use", destination: AppLinks.termsOfUse)
                    Link("Support", destination: AppLinks.support)
                    LabeledContent("Version", value: appVersion)
                }

                Section {
                    Text("SoleCount keeps all your data on this iPhone. No account, no tracking,
no ads.")
                        .font(.footnote)
                        .foregroundStyle(.secondary)
                }
            }
            .navigationTitle("Settings")
            .navigationBarTitleDisplayMode(.inline)
            .toolbar {
                ToolbarItem(placement: .confirmationAction) {
                    Button("Done") { dismiss() }
                }
            }
```

```
            .sheet(isPresented: $showingPaywall) { PaywallView() }
            .alert("Store", isPresented: $store.showError) {
                Button("OK", role: .cancel) { }
            } message: {
                Text(store.errorMessage)
            }
        }
    }
}
```

5.11 PrivacyInfo.xcprivacy (privacy manifest)

Tells Apple the app doesn't track you or collect data, and explains why it uses `UserDefaults` (reason **CA92.1**: to remember whether Pro is unlocked).

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
	<key>NSPrivacyTracking</key>
	<false/>
	<key>NSPrivacyTrackingDomains</key>
	<array/>
	<key>NSPrivacyCollectedDataTypes</key>
	<array/>
	<key>NSPrivacyAccessedAPITypes</key>
	<array>
		<dict>
			<key>NSPrivacyAccessedAPIType</key>
			<string>NSPrivacyAccessedAPICategoryUserDefaults</string>
			<key>NSPrivacyAccessedAPITypeReasons</key>
			<array>
				<string>CA92.1</string>
			</array>
		</dict>
	</array>
</dict>
</plist>
```

5.12 CLAUDE.md (in ~/Documents/SoleCount, next to SoleCount.xcodeproj)

Instructions Claude Code reads automatically if you use it later to add features.

```
# SoleCount — project guide for Claude Code

## What this app is
SoleCount is an iPhone app that tracks how often I wear each pair of sneakers
and shows the cost per wear (price ÷ number of wears).

## Tech rules (always follow)
- SwiftUI + SwiftData + Swift Charts + StoreKit 2. Minimum iOS 18.
- No servers, no accounts, no analytics, no ads, no third-party SDKs or packages.
- All data stays on the device. Do not add networking code.
- One main type per file. Keep files short and readable.
```

```
- Use @Observable (not ObservableObject). Use NavigationStack.
- Build must compile with zero errors before you say you are done.
- Explain every change in plain English. I am a beginner.
- Do not rename the bundle ID or the product ID.

## Files
- SoleCountApp.swift — app entry, creates StoreManager + SwiftData container
- Sneaker.swift — @Model: name, brand, price, purchaseDate, createdAt,
  photoData (external storage), wearDates [Date], isRetired, wearLimit Int?
- SneakerListView.swift — list (In rotation / Retired), add button, free limit
- AddSneakerView.swift — form with PhotosPicker, price, date, wear limit
- SneakerDetailView.swift — stats, big "Wore today" button, undo, retire, chart
- WearChartView.swift — bar chart of wears per month (last 6 months)
- StoreManager.swift — StoreKit 2, one non-consumable
- PaywallView.swift — Pro screen with Buy + Restore Purchases + links
- SettingsView.swift — Pro status, Restore Purchases, privacy policy, version
- Helpers.swift — currency formatting, photo shrinking, SneakerPhoto, StatCard
- PrivacyInfo.xcprivacy — no tracking, no data collected, UserDefaults CA92.1

## Business rules
- Free: up to 3 pairs (StoreManager.freePairLimit). Chart is Pro only.
- Pro: one-time non-consumable, product ID `com.krishnav.SoleCount.pro`, $4.99.
- Paywall must always show: price from StoreKit (displayPrice), Restore Purchases,
  Privacy Policy and Terms links. Never hard-code the price in text.

## App Store rules to respect
- No claims we can't back up. No fake reviews prompts. No "free" if it isn't.
- Restore Purchases must stay visible in Settings and the paywall.
- If you add any new "required reason" API (UserDefaults, file timestamps,
  disk space, system boot time), update PrivacyInfo.xcprivacy.

## Testing
- Local purchases use SoleCount.storekit (Scheme > Run > Options > StoreKit Configuration).
- Test on the iPhone 17 simulator and on my real iPhone.
```

5.13 .gitignore (in ~/Documents/SoleCount)

Tells Git to skip personal Xcode settings and build junk. The name starts with a dot, which makes it a hidden file. In Finder, press **Cmd+Shift+.** (period) to show hidden files.

```
# macOS
.DS_Store

# Xcode user-specific files
xcuserdata/
*.xcuserstate
*.xcscmblueprint
*.xccheckout

# Build output
build/
DerivedData/
*.ipa
*.dSYM.zip
*.dSYM
```

```
# Swift Package Manager
.build/
.swiftpm/

# Secrets (never commit these)
*.p8
*.p12
*.mobileprovision
.env
```

5.14 PRIVACY.md (in ~/Documents/SoleCount)

Your privacy policy. Apple requires a public privacy policy link. Because your GitHub repo is public, this file's GitHub page works as that link (the app already points to it).

```
# SoleCount Privacy Policy

_Last updated: November 2026_

SoleCount does not collect, store, or share any personal data.

- Everything you enter (sneakers, photos, prices, wear history) is stored only on your iPhone.
- SoleCount has no accounts, no analytics, no advertising and no tracking.
- Photos you choose are copied into the app's own storage on your device and are never uploaded.
- Purchases are handled by Apple through the App Store. We never see your payment details.
- If you delete the app, all of its data is deleted from your device.

Questions? Open an issue at https://github.com/KrishnaDistributedcomputing/SoleCount/issues
```

Step 6. Build the app

1. In Xcode, press **Cmd+Shift+K** (**Product > Clean Build Folder**).
2. Press **Cmd+B** (**Product > Build**).
3. Wait for **Build Succeeded** at the top of the window.

If you see red errors, read the **Troubleshooting** section at the end. The most common cause is a missing first or last line when pasting, or a file that was pasted twice.

Part 3: Run the app

Step 7. Run it on the simulator

1. At the top center of Xcode, click the device name (it may say **My Mac** or **Any iOS Device**) and choose an iPhone simulator, such as **iPhone 17**.
2. Press the ▶ **Run** button (or **Cmd+R**). The simulator opens and SoleCount starts.
3. Try it out:
    - Tap **+**, add a pair (photo, name, brand, price), and tap **Save**.
    - Tap the pair, then tap **Wore today**. Wears goes to 1, and cost per wear equals the price.
    - Add 3 pairs, then tap **+** again. The **SoleCount Pro** screen should appear. That's the free limit working.
    - On a pair's screen, tap **Retire**. It moves to the **Retired** section.
    - Swipe left on a pair in the list to delete it.
4. Press the ■ **Stop** button when you're done.

> The simulator's Photos app has a few sample pictures. To use your own, drag an image file from Finder onto the simulator window.

Step 8. Set up test purchases (StoreKit configuration file)

This lets you "buy" Pro for free while testing, with no App Store account needed yet.

1. In Xcode, click the top **SoleCount** folder in the left panel.
2. **File > New > File from Template…**, type `storekit` in the filter box, choose **StoreKit Configuration File**, and click **Next**.
3. Name it `SoleCount`. Leave **Sync this file with an app in App Store Connect unchecked**. Click **Create**.
4. The file `SoleCount.storekit` opens. Click the **+** button at the bottom left and choose **Add Non-Consumable In-App Purchase**.
5. Fill in:
    - **Reference Name:** `SoleCount Pro`
    - **Product ID:** `com.krishnav.SoleCount.pro` (must match exactly, including capital S and C)
    - **Price:** `4.99`
    - Under **Localizations**, click **+** (or double-click the English row): **Display Name** `SoleCount Pro`, **Description** `Unlimited pairs and wear history chart`
6. Press **Cmd+S**.
7. Tell Xcode to use it: **Product > Scheme > Edit Scheme…** > click **Run** on the left > **Options** tab > **StoreKit Configuration** > choose **SoleCount.storekit** > **Close**.

Step 9. Test the purchase

1. Press ▶ **Run**.
2. Tap **+** until the Pro screen appears (after 3 pairs), or tap the gear icon and choose **Unlock Pro**.
3. The button should say **Unlock Pro for $4.99**. Tap it and confirm the test purchase.
4. The screen closes. You can now add a 4th pair, and each pair shows its chart.
5. **To test again from zero:** in Xcode choose **Debug > StoreKit > Manage Transactions…**, select the purchase, and click the **Delete** (trash) button. Then stop and run the app again.
6. **Test Restore:** after deleting the purchase, buy it again, then tap **Settings > Restore Purchases**. It should say Pro is unlocked.

> If the button says **Loading price…** forever, the scheme isn't using the StoreKit file (repeat item 7 in Step 8) or the Product ID has a typo.

Step 10. Run it on your own iPhone (free Apple ID is fine)

1. **Sign in to Xcode: Xcode > Settings > Accounts** > click **+** > **Apple ID** > sign in with your Apple ID.
2. **Pick your team:** In the left panel click the blue **SoleCount** project icon at the top, then under **TARGETS** click **SoleCount** and open the **Signing & Capabilities** tab. Make sure **Automatically manage signing** is ticked, then set **Team** to **Krishna Venkataraman (Personal Team)**.
    - If it complains that the bundle ID isn't available, change **Bundle Identifier** to something unique like `com.krishnav.SoleCount2` for testing. (Change it back before publishing, and keep the Pro product ID as `com.krishnav.SoleCount.pro`.)
3. **Connect the iPhone** with the cable. Unlock it and tap **Trust** on the phone, then enter your passcode.
4. **Turn on Developer Mode** on the iPhone: **Settings > Privacy & Security > Developer Mode** (scroll to the bottom) > turn it on > **Restart**. After the restart, tap **Turn On** and enter your passcode. (The option only appears after the phone has been connected to Xcode.)
5. In Xcode's device menu at the top, choose **your iPhone** (not a simulator), then press ▶ **Run**.
6. **First time only:** if the phone says **Untrusted Developer**, go to **Settings > General > VPN & Device Management**, tap your Apple ID under **Developer App**, and tap **Trust**. Run again from Xcode.

> With a free Apple ID, apps on your phone stop opening after **7 days**. Just run it from Xcode again. Once you join the paid Apple Developer Program ($99 USD/year), this limit goes away.

Part 4: Put it on GitHub

Your empty repo already exists: **https://github.com/KrishnaDistributedcomputing/SoleCount** (username **KrishnaDistributedcomputing**). Now you'll send your project to it. Everything below is typed in **Terminal**.

Step 11. Check that Git is installed

```
git --version
```

You should see something like `git version 2.x`. If a window asks to install "command line developer tools", click **Install**, wait, then run the command again.

Step 12. Tell Git who you are (one time only)

Use the email address of your GitHub account:

```
git config --global user.name "Krishna Venkataraman"
git config --global user.email "YOUR-GITHUB-EMAIL@example.com"
```

Step 13. Go to the project and check for an existing repository

```
cd ~/Documents/SoleCount
git status
```

- If you see `On branch main` (or `master`), Xcode already created a Git repository for you, because the "Create Git repository on my Mac" box is ticked by default. Go to Step 14.
- If you see `fatal: not a git repository`, create one now:

```
git init
```

Step 14. Make sure .gitignore is there

```
ls -a
```

You should see `.gitignore` in the list. If not, copy it from the package:

```
cp ~/Downloads/SoleCount-Package/.gitignore .
```

If Xcode had already committed personal settings files, stop tracking them (it's safe if this prints an error saying nothing matched):

```
git rm -r --cached SoleCount.xcodeproj/xcuserdata
SoleCount.xcodeproj/project.xcworkspace/xcuserdata
```

Step 15. Save a snapshot (commit)

```
git add .
git commit -m "SoleCount v1: sneaker list, wear tracking, Pro purchase"
git branch -M main
```

- `git add .` picks up every file (except those ignored by .gitignore).
- `git commit` saves the snapshot with a message.
- `git branch -M main` names your branch `main`, the same name GitHub uses.

Step 16. Connect your Mac's project to your GitHub repo

```
git remote add origin https://github.com/KrishnaDistributedcomputing/SoleCount.git
git remote -v
```

The second command should show that URL twice (fetch and push).

> If you see `error: remote origin already exists`, run this instead: `git remote set-url origin https://github.com/KrishnaDistributedcomputing/SoleCount.git`

Step 17. Sign in to GitHub from Terminal

GitHub doesn't accept your normal password in Terminal. Pick **one** of these:

Option 1 (easiest): GitHub CLI

1. Install Homebrew if you don't have it (paste the command from **https://brew.sh** into Terminal and follow its instructions, including the two "Next steps" lines it prints at the end).
2. Install the GitHub CLI and log in:

```
brew install gh
gh auth login
```

3. Answer the questions: **GitHub.com** > **HTTPS** > **Yes** (authenticate Git with your GitHub credentials) > **Login with a web browser**. Copy the one-time code, press **Return**, paste the code in the browser page, and approve.

Option 2: Personal access token

1. On github.com, click your profile picture > **Settings** > **Developer settings** (at the bottom of the left menu) > **Personal access tokens** > **Tokens (classic)** > **Generate new token (classic)**.
2. Note: `SoleCount Mac`. Expiration: 90 days. Tick the **repo** box. Click **Generate token**.
3. **Copy the token now** (it starts with `ghp_`). You won't see it again.
4. In Step 18, when Terminal asks for a **Username**, type `KrishnaDistributedcomputing`. When it asks for a **Password**, paste the token. Nothing shows while you paste, which is normal. Press **Return**. Your Mac's

Keychain remembers it for next time.

Step 18. Upload (push) to GitHub

Case A: the repo is empty (this is your case). When the repo was checked on 2 Nov 2026, it had no files, no README and no license. Just push:

```
git push -u origin main
```

Case B: the repo has a README or license that GitHub created. The first push is then rejected with `rejected … (fetch first)`, because GitHub has a commit your Mac doesn't. Get it first, then push:

```
git pull origin main --allow-unrelated-histories
git push -u origin main
```

- If Git opens a text editor asking for a merge message, type `:wq` and press **Return** (that saves and closes it).
- If it says a file like README.md has a **conflict**, open the file, keep the text you want, delete the `<<<<<<<`, `=======` and `>>>>>>>` lines, then run `git add README.md`, `git commit -m "Merge GitHub README"` and `git push -u origin main`.
- Alternative: `git pull --rebase origin main` instead of the first line gives a tidier history. Either one works.

Check it worked: open **https://github.com/KrishnaDistributedcomputing/SoleCount** in your browser and refresh. You should see your files, including the `SoleCount` folder, `SoleCount.xcodeproj`, `README.md` and `PRIVACY.md`.

> **Your repo is public**, so anyone can see the code. That's fine for learning, and it lets the PRIVACY.md page work as your App Store privacy policy link. To make it private: on GitHub, go to **Settings > General > Danger Zone > Change visibility**. If you do, host the privacy policy somewhere public, such as a public GitHub Gist, and update `AppLinks.privacyPolicy` in SettingsView.swift.

Step 19. Every time you change something later

```
cd ~/Documents/SoleCount
git status
git add .
git commit -m "Describe what you changed"
git push
```

`git status` shows what changed. The commit message should say what you did, for example `Add dark mode colors`.

Step 20. Or use Xcode's built-in Git buttons instead of Terminal

1. **Xcode > Settings > Accounts > + > GitHub**. Enter `KrishnaDistributedcomputing` and a **personal access token** (from Step 17, Option 2. Xcode asks for the scopes it needs: **repo**, plus **admin:public_key**, **write:discussion** and **user**).
2. To commit: use the **Integrate** menu (called **Source Control** in older Xcode) > **Commit…**. Tick the files, type a message at the bottom, and click **Commit**.
3. To upload: **Integrate > Push…** > choose **origin/main** > **Push**.
4. You can also open the **Source Control navigator** (press **Cmd+2**) to see branches and history.

Part 5: Optional — let Claude Code add features

Your `CLAUDE.md` is already in the project folder, so Claude Code knows the rules (no servers, no SDKs, keep Restore Purchases, and so on).

```
cd ~/Documents/SoleCount
claude
```

Example prompts you can paste:

- `Read CLAUDE.md. Add a "Share my rotation" button on the list screen that creates an image of my top 3 pairs by wears and opens the share sheet. Keep it free. Build must compile.`
- `Read CLAUDE.md. Add a home screen widget that shows my most-worn pair. Make it a Pro feature. Explain each step I must do in Xcode by hand (like adding the widget target and App Group).`
- `Read CLAUDE.md. Add an "Edit pair" screen so I can change name, brand, price, photo and wear limit.`

After Claude finishes, press ▶ **Run** in Xcode to test, then commit and push (Step 19).

Part 6: Getting ready for the App Store (summary)

1. **Join the Apple Developer Program** ($99 USD/year, once for all your apps): **developer.apple.com/programs**. Then join the **App Store Small Business Program**, so Apple takes 15% instead of 30%.
2. **Sign the Paid Apps Agreement:** in **App Store Connect > Business**, add your bank and tax info. Without it, the purchase won't work in the real store.
3. **Create the app** in App Store Connect: **Apps > + > New App**, with bundle ID `com.krishnav.SoleCount`.
4. **Create the in-app purchase: Monetization > In-App Purchases > +** > **Non-Consumable**, Product ID `com.krishnav.SoleCount.pro`, price $4.99, display name and description, and a **review screenshot** of your Pro screen. Attach it to your first app version when you submit.
5. **App Privacy:** answer **"Data Not Collected"**. Privacy Policy URL: `https://github.com/KrishnaDistributedcomputing/SoleCount/blob/main/PRIVACY.md`.
6. **Screenshots:** take them on your iPhone or the simulator (**Cmd+S** in the simulator). Use real app screens, not mockups. You need at least one 6.9" or 6.3" size (e.g. 1320×2868, 1206×2622 or 1179×2556 pixels).
7. **Upload:** set the device menu to **Any iOS Device (arm64)**, then **Product > Archive** > **Distribute App** > **App Store Connect** > **Upload**. When asked about encryption, choose **None of the algorithms mentioned above** (the app only uses Apple's standard HTTPS through StoreKit).
8. **Review notes** (paste this in "Notes" for App Review): *"SoleCount stores all data on-device. Free users can track 3 pairs; the one-time non-consumable 'SoleCount Pro' unlocks unlimited pairs and the wear-history chart. Restore Purchases is in Settings and on the Pro screen. No account or login is required."*
9. **Age rating:** answer **None** to everything, which gives **4+**.

Why this should pass review: it does something real and useful (Guideline 4.2), uses Apple's In-App Purchase for the unlock (3.1.1), has Restore Purchases (3.1.1), collects no data and has a privacy policy (5.1.1), and has no login or third-party SDKs.

Troubleshooting

What you see	What it means and how to fix it
`Cannot find 'Item' in scope` or `Cannot find 'ContentView' in scope`	SoleCountApp.swift still has the old template code. Replace all of it with the code in 5.1.
`Cannot find 'SneakerListView'` (or another type) `in scope`	That file is missing or not in the app target. Check that it's in the left panel. Click it, open the right panel (**Cmd+Option+0**) and tick **Target Membership: SoleCount**.
`Invalid redeclaration of 'SneakerPhoto'` (or similar)	The same code was pasted into two files. Each type should exist in only one file.
`Expected '}'` or `Extraneous '}'`	A bracket got lost while pasting. Select all in that file, delete, and paste the file again.
`Multiple commands produce … PrivacyInfo.xcprivacy`	There are two PrivacyInfo files. Delete one.
Lots of concurrency warnings or errors mentioning `MainActor` or `Sendable`	Click the blue project icon > **TARGETS SoleCount** > **Build Settings** > search `Swift Language Version` and set it to **Swift 5**. Also check that **Default Actor Isolation** is **MainActor**, then clean and build.
Pro button says **Loading price…** forever	In the simulator: the scheme isn't using `SoleCount.storekit` (Step 8, item 7), or the Product ID doesn't match `com.krishnav.SoleCount.pro`. On TestFlight or App Store: the IAP isn't created yet, or the Paid Apps Agreement isn't signed.
App crashes on launch after you changed `Sneaker.swift`	The old saved data no longer matches. Delete the app from the simulator or iPhone (long-press the icon > Remove App) and run again.
`Signing for "SoleCount" requires a development team`	Do Step 10, item 2.
`Failed to register bundle identifier`	That bundle ID is taken for free accounts. Use `com.krishnav.SoleCount2` while testing.
iPhone not shown in the device menu	Unlock the phone, tap **Trust**, turn on Developer Mode, try another cable, or use **Window > Devices and Simulators**.
`Untrusted Developer` on the iPhone	**Settings > General > VPN & Device Management** > your Apple ID > **Trust**.
Only **My Mac** or **Any iOS Device** shows	The iOS simulator isn't installed yet. Go to **Xcode > Settings > Components** and download **iOS**.
`git: 'credential-osxkeychain'…` or `Authentication failed`	GitHub needs a token, not your password. Use Step 17 (gh CLI or token).
`remote: Repository not found`	Check for typos in the URL, and that you're logged in as KrishnaDistributedcomputing: `git remote -v`.
`! [rejected] main -> main (fetch first)`	GitHub has files your Mac doesn't. See Step 18, Case B.
`error: src refspec main does not match any`	You haven't committed yet. Run Step 15 first.
`fatal: refusing to merge unrelated histories`	Add `--allow-unrelated-histories` to the pull (Step 18, Case B).

What you see	What it means and how to fix it
Push shows `xcuserdata` files on GitHub	Run the `git rm -r --cached …` line in Step 14, then commit and push.

Still stuck? Take a screenshot of the error (**Cmd+Shift+4**, then drag) and send it to me.